

Taller 10

Grupo 6 - Sistema TicketBeat
Diseño de Software

Integrantes:

Pablo Ojeda
Daniel Negrete
David Matos
Fernando Zambrano

JUSTIFICACIÓN

PÁRRAFO 1 — Patrón Factory Method (Fernando Zambrano)

TicketBeat maneja múltiples tipos de boletos (asientos reservados, entradas generales, pases VIP), cada uno con lógica de negocio, accesos y precio base distintos. Se define una clase abstracta `CreadorBoleto` con un método de fabricación (`crearBoleto()`), delegando la instanciación a subclases concretas (`CreadorBoletoVIP`, `CreadorBoletoGeneral`, `CreadorBoletoAsientoReservado`), cada una encapsulando la inicialización de su variante. Este patrón aísla la instanciación cumpliendo el principio Abierto/Cerrado: agregar un nuevo tipo (ej. "Meet & Greet") solo requiere una nueva subclase, sin modificar el flujo de compra ni usar `if/else`. Factory Method es adecuado porque existe una única jerarquía de producto (`Boleto`) con variantes, no familias de objetos relacionados que deban coordinarse entre sí.

PÁRRAFO 2 — Patrón Abstract Factory (David Matos)

TicketBeat permite buscar eventos y reservar boletos antes de pagar, procesando el cobro automáticamente mediante distintas formas de pago (tarjeta, transferencia, pago móvil), pensado para implementarse en web y móvil. Se define una interfaz `FabricaFlujoPago` con métodos para crear los componentes de cobro de cada plataforma, y fábricas concretas (`FabricaFlujoWeb`, `FabricaFlujoMobile`) que garantizan consistencia entre los componentes producidos según la plataforma. Este patrón es adecuado porque el enunciado exige coordinar una familia de objetos relacionados por plataforma, evitando mezclar componentes de una plataforma con otra. Cada fábrica concreta puede delegar en estrategias de pago (`PagoTarjeta`, `PagoTransferencia`, `PagoMovil`) para encapsular la lógica de cada método, pero la estructura principal que resuelve el requisito móvil/web es Abstract Factory.

PÁRRAFO 3 — Patrón Facade (Daniel Negrete)

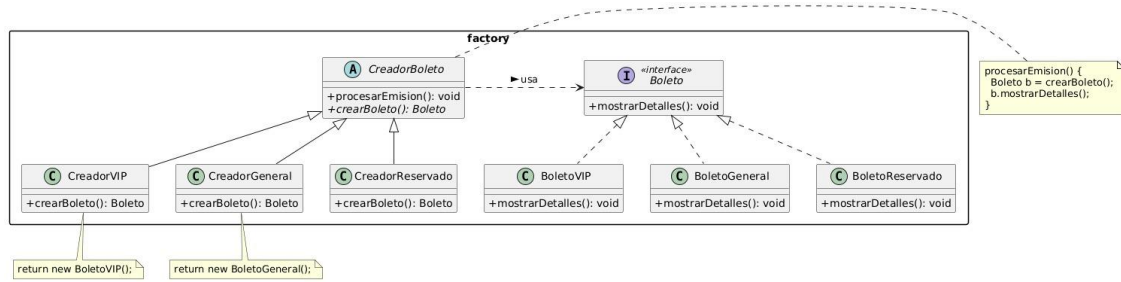
TicketBeat expone un flujo con múltiples subsistemas: `BuscadorEventos`, `GestorReservas`, `ProcesadorPagos` y `GestorNotificaciones`. Sin una fachada, la interfaz (web/móvil) tendría que conocer y coordinar directamente estos subsistemas, generando alto acoplamiento. Se aplica Facade mediante `TicketBeatFacade`, que expone un único punto de acceso para ejecutar el proceso completo (buscar → reservar → pagar → notificar), orquestando internamente el orden correcto sin exponer los detalles de cada subsistema al cliente. Internamente usa `CreadorBoleto` (Factory Method) y `FabricaFlujoPago` (Abstract Factory), integrando los patrones creacionales bajo una interfaz simple y coherente.

PÁRRAFO 4 — Patrón Chain of Responsibility (Pablo Ojeda)

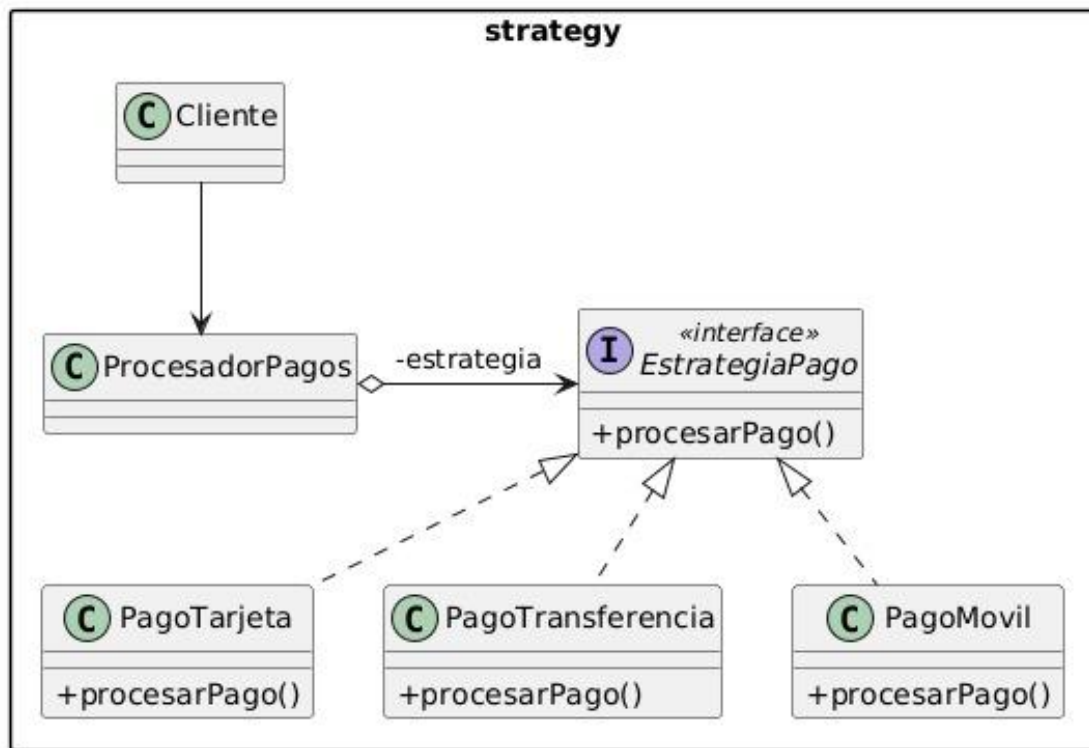
El caso presenta una cadena de manejadores Soporte -> Administración, donde: el cliente (usuario) genera una solicitud (incidente) sin conocer qué manejador la resolverá finalmente (desacoplamiento cliente-receptor); cada manejador (agente de soporte, luego administración) recibe la solicitud y decide autónomamente si la resuelve o la reenvía al siguiente eslabón; y la solicitud se propaga secuencialmente hasta encontrar un manejador que la resuelva (administración, en última instancia).

DIAGRAMAS DE CLASES

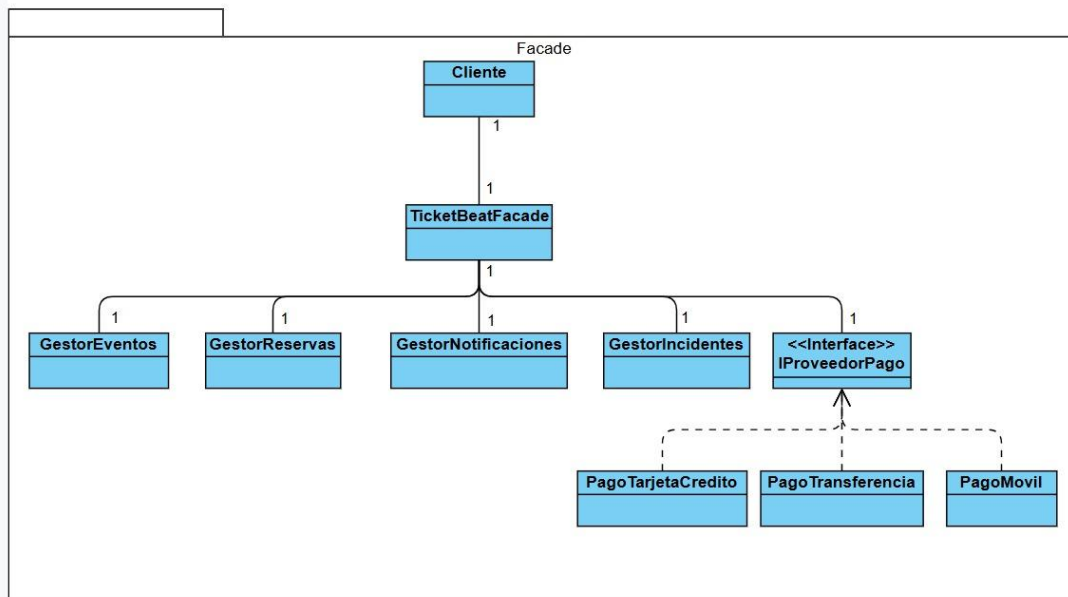
Factory Method



David Matos



Estrategy



Chain of Responsibility

